## Vectorized Implementation in Practice (Optional)

This section connects the theory of matrix multiplication directly to the efficient, vectorized code used to implement a neural network layer.

### The Goal: Replacing Loops with Matrix Multiplication

Instead of using a `for` loop to calculate the activation of each neuron one by one, we can use a single matrix multiplication operation to compute the activations for the *entire layer* at once.

---

### Setting Up the Matrices

To use matrix multiplication, all our data (inputs, weights, and biases) must be 2D arrays (matrices).

1. **Input Activations (`A_in` or `X`):**
    - The input features for a single example are represented as a **row vector** (a 1x`n` matrix, where `n` is the number of features).
    - `X = np.array([[200, 17]])` (This is a 1x2 matrix)
2. **Weights (`W`):**
    - The weight vectors for all neurons in the *current* layer are stacked together as **columns**.
    - If Layer 1 has 3 neurons and 2 input features, `W` will be a **2x3 matrix**.
    - `W = np.array([[w1_1, w1_2, w1_3], [w2_1, w2_2, w2_3]])`
3. **Biases (`B`):**
    - The biases for all neurons in the current layer are stored in a **row vector**.
    - `B = np.array([[b_1, b_2, b_3]])` (This is a 1x3 matrix)

---

### The Vectorized Computation 🚀

The entire `for` loop from the "from-scratch" implementation is replaced by two lines of code:

1. **Compute `Z` (the linear part):**
    - **`Z = np.matmul(X, W) + B`**
    - This single `np.matmul(X, W)` operation performs the dot product of the input row vector `X` with *each column* of `W`, producing a 1x3 row vector.
    - The bias vector `B` is then added to this result.
    - The resulting `Z` is a 1x3 matrix containing all the $z$ values for the layer: `[[z_1, z_2,`

`z_3]]`.

2. **Compute `A_out` (the activation):**
    - **`A_out = g(Z)`**
    - The sigmoid activation function `g` is applied *element-wise* to the `Z` matrix.
    - The result `A_out` is a 1x3 matrix with the final activations for the layer: `[[a_1, a_2, a_3]]`.

### Why This is So Powerful

This vectorized approach is the key to deep learning's performance.

- **Efficiency:** Modern hardware (CPUs and especially GPUs) is highly optimized to perform matrix multiplications (`matmul`) incredibly fast, far faster than a Python `for` loop.
- **Simplicity:** The code becomes much cleaner and more compact.

**Note on Syntax:** In some Python code, you may also see matrix multiplication written with the `@` operator: `Z = X @ W + B`. This is an alternative syntax for `np.matmul(X, W)`.